Workload-Aware Torch-TensorRT Configuration Selection

Section draft for the team report (Experiment E4)

Tanish Vardhineni
New York University

I. Workload-Aware Configuration Selection

The previous experiments establish that Torch-TensorRT latency depends on how the graph is partitioned and on the shape of the request. This experiment asks the question that follows from both: if the best compiler configuration moves with the workload, can a cheap model learn where it moves, so that a deployment picks its configuration from the batch size and sequence length instead of committing to one setting for everything? Rice's algorithm selection problem [1] gives the framing, and cost-model-driven autotuners such as AutoTVM [2] show that the idea pays off when the configuration space is large and the optimum is workload dependent.

Our answer for DistilBERT on an A100 is negative, and we report it as such. A single tuned configuration is within 0.5% of the best tested configuration on every held-out shape. The two selectors we built reproduce that configuration almost everywhere and, where they deviate, they mostly lose. What does matter is tuning at all: the tuned configuration is $1.11\times$ faster than default Torch-TensorRT on the unmodified model and $1.6\times$ to $2.3\times$ faster once part of the graph falls back to PyTorch. Getting to a result we trust took three iterations, and two of them changed the experiment more than the selector did. We describe them in the order they happened, because the intermediate findings (a model with nothing to partition, a cluster that cancels compile-heavy jobs, and a 20% timing artifact) are useful to the rest of the project on their own.

A. Problem Formulation

A workload is a request shape $w = (b, s)$, with batch size b and sequence length s in tokens. A configuration $c \in \mathcal{C}$ is one way of running the model: PyTorch eager, or Torch-TensorRT with a particular set of compiler and runtime settings. $L(c, w)$ is the warmed-up median latency of c on w. A selector is a function $\pi : w \mapsto c$ fitted on measurements from a set of training shapes W_{train} and judged on shapes it has not seen. For a test set W we report the *regret* of π against the best tested configuration,

$$R(\pi) \;=\; \Big(\prod_{w \in W} \frac{L(\pi(w), w)}{\min_{c \in \mathcal{C}} L(c, w)} \Big)^{1/|W|} - 1, \qquad (1)$$

that is, the geometric-mean slowdown relative to an oracle that tries every configuration on every shape. A geometric mean keeps a 5% loss at batch 1 and a 5% loss at batch 16 on equal footing, although the second is ten times larger in milliseconds.

We compare six methods, following the evaluation plan of the proposal:

- **PyTorch eager**, the uncompiled FP16 model;
- **Default Torch-TensorRT**, the compiler's default settings;
- **One fixed tuned configuration**, the configuration with the lowest geometric-mean slowdown over W_{train}, used for every shape;
- **Lookup selector**, which returns the best configuration of the nearest training shape, with distance measured in $(\log_2 b, \log_2 s)$;
- **Regression selector**, which fits one latency model per configuration and returns the configuration with the lowest prediction (Section I-F);
- **Best tested configuration**, the oracle in (1).

The fixed tuned configuration is the baseline that decides the experiment. A selector is only worth deploying if it beats the best single configuration, not merely the default one.

B. Experimental Setup

All measurements come from one NVIDIA A100-SXM4-40GB node of the NYU HPC cloud bursting cluster, inside a Singularity container with PyTorch 2.14.1 (CUDA 13.0) [3], Torch-TensorRT 2.14.0 [4] and TensorRT 11.1.0 [5]. The model is `distilbert-base-uncased` [6] in FP16, wrapped so that it takes token ids and an attention mask and returns the last hidden state. Inputs are random token ids from a fixed seed with a full attention mask, and every configuration receives the same tensors. Torch-TensorRT uses its Dynamo front end, and each engine is built for one static shape, so a (configuration, shape) pair is one compilation.

The training grid is $b \in \{1, 4, 8, 16\} \times s \in \{128, 256, 512\}$, twelve shapes. The proposal also lists 1024 tokens, but DistilBERT has 512 position embeddings, so longer inputs cannot be run without changing the model. Eight further shapes lie off the grid and are never used for fitting: $(1, 384)$, $(2, 192)$, $(2, 384)$, $(4, 320)$, $(8, 192)$, $(12, 192)$, $(12, 384)$ and $(16, 448)$. None of their sequence lengths occurs in the grid, and four of them also use a batch size (2 or 12) that does not.

Latency is measured with the project's shared benchmark routine: 20 warm-up calls followed by 100 calls timed with CUDA events, from which we take the median and the 95th percentile. Around it, our sweep harness records the compile time, the time of the first call, the number of TensorRT and

TABLE I
CHRONOLOGY OF CLUSTER RUNS FOR THIS EXPERIMENT (ALL ON 12 NOV. 2026)

Job	Purpose	Outcome
29499	Environment	Cancelled: `--fakeroot` unpacked the 8.5 GB image to network storage
29507	Environment	Held by the scheduler; resubmitted
29509	Environment	Packages installed (plain writable overlay)
29537	Partition check	One engine, no fallback; LayerNorm exclusion had no effect (wrong operator name)
29550	Partition check	Corrected; Table II
29551	Sweep 1	GPU node never booted; cancelled after 12 min
29586	Sweep 1	83 of 340 runs, then cancelled by the cluster's low-utilization watchdog at 30 min
29727–8	Sweep 2	Two more node boot failures (13 min each)
29729	Sweep 2	340 of 340 runs in 2 h 31 min; all results below

PyTorch blocks in the compiled module, and the agreement of the output with eager PyTorch (cosine similarity and largest absolute difference). The harness appends one row per timing repeat to a CSV file and skips pairs that are already present, so an interrupted job resumes where it stopped. That property turned out to be necessary (Section I-D).

Table I lists every cluster job behind this section. Only the last one contributes numbers to the results; the others are the reason the last one looks the way it does.

C. Iteration 1: Is There Anything to Tune?

Our plan assumed that the partitioning controls of Torch-TensorRT (`min_block_size`, the choice between the fast and the global partitioner, and operator exclusion) would produce visibly different graphs for DistilBERT, and that the selector would arbitrate between them. Before spending GPU hours on engines we checked that assumption with a dry-run compile of every candidate at $(4, 128)$, which reports the partition without building anything.

The assumption was wrong. After lowering, DistilBERT consists of 127 operators (36 linear layers, 24 reshapes, 24 permutes, 13 additions, 13 layer normalizations, six attention and six GELU operators, and a handful of embedding and mask operations), and TensorRT supports all of them. The default compile is therefore a single engine with no PyTorch fallback, and `min_block_size` and the partitioner have nothing to act on: we verified that `min_block_size` $= 1$ and the global partitioner return the identical one-engine graph, and removed both from the sweep.

That left two options: switch to a model with unsupported operators, which would have cut this experiment off from the shared harness and from the other three experiments, or keep the model and create the partitioning problem deliberately. We chose the second and say so plainly, because it limits what the results mean (Section I-I). Torch-TensorRT lets the user force named operators to run in PyTorch. We define three *scenarios*:

TABLE II
PARTITION STRUCTURE OF EACH CONFIGURATION (DRY-RUN COMPILE)

Configuration	Blocks TRT	Blocks PyT.	Operators per TRT block
`trt_default`, `trt_opt*`	1	0	127
`gelu_mbs5`, `gelu_global`	6	6	23, then 5×19
`gelu_mbs1`	7	6	23, 5×19, 3
`gelu_mbs22`	1	1	23 (104 ops in PyTorch)
`ln_mbs5`	7	7	6, then 6×14
`ln_mbs1`	13	13	6, then $6 \times (14, 4)$
`ln_mbs10`	6	7	6×14
`ln_mbs20`	0	1	none (127 ops in PyTorch)
`ln_global`	6	7	18, then 5×14

- **Unmodified model**: nothing excluded, one engine.
- **GELU in PyTorch**: the six GELU operators are excluded, which splits the encoder between its layers.
- **LayerNorm in PyTorch**: the thirteen layer normalizations are excluded, which cuts every layer twice.

Each scenario is evaluated separately, with its own default, its own tuned configuration and its own selectors, as if it were a different model.

Table II shows the resulting partitions. The dry run also pruned the configuration list a second time. With GELU excluded, the TensorRT blocks hold 19 to 23 operators, so `min_block_size` values of 5, 10 and 20 all give the same graph; we kept 5 and replaced the other two by 22, the smallest value that changes anything. Two mistakes surfaced in the same pass. Our first LayerNorm scenario excluded `aten.layer_norm`, but the operator that exists after lowering is `aten.native_layer_norm`; the exclusion silently matched nothing and the "broken" model was still one engine. And our block counter reported zero PyTorch blocks for the global partitioner, because that partitioner leaves fallback operators inline in the top-level graph instead of wrapping them in submodules. Both were fixed before any latency was measured. Without the dry run, a third of the sweep would have been mislabeled.

The final space has 17 configurations (Table III). Because partitioning offers nothing on the unmodified model, that scenario varies the builder's optimization level instead. All three scenarios include whole-graph CUDA Graphs [7], a runtime option that records the module's stream of GPU kernel launches once and replays it as a single submission. One more configuration, `trt_default_rebuild`, is the default built a second time. It belongs to no scenario and exists to measure how much two builds of the same configuration differ.

D. Iteration 2: A First Sweep That Could Not Be Used

The first sweep timed each compiled module three times with the shared protocol. It was cancelled after 30 minutes with 83 of 340 runs done, and when we looked at those 83 runs they were not usable either. The two problems are unrelated.

1) Compile-heavy jobs are cancelled: The accounting record showed the job cancelled by the administrator account

TABLE III
CONFIGURATIONS IN THE SWEEP. DEFAULT SETTINGS ARE `MIN_BLOCK_SIZE` = 5, FAST PARTITIONER, OPTIMIZATION LEVEL 3.

Scenario	Configuration	Change from scenario default
all	`eager`	PyTorch, no compilation
Unmodified	`trt_default`	(default)
	`trt_opt1`	optimization level 1
	`trt_opt5`	optimization level 5
	`trt_cudagraphs`	CUDA Graphs at run time
	`trt_default_rebuild`	none (second build)
GELU in PyTorch	`gelu_mbs5`	(default)
	`gelu_mbs1`	`min_block_size` 1
	`gelu_mbs22`	`min_block_size` 22
	`gelu_global`	global partitioner
	`gelu_cudagraphs`	CUDA Graphs at run time
LayerNorm in PyTorch	`ln_mbs5`	(default)
	`ln_mbs1`	`min_block_size` 1
	`ln_mbs10`, `ln_mbs20`	`min_block_size` 10, 20
	`ln_global`	global partitioner
	`ln_cudagraphs`	CUDA Graphs at run time

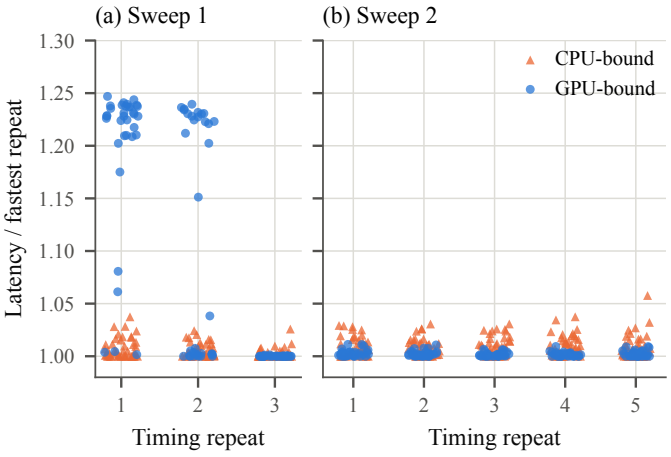


Fig. 1. Each timing repeat relative to the fastest repeat of the same compiled module, for the five shapes both sweeps cover. (a) With the shared 20-iteration warm-up, GPU-bound configurations start about 20% slow and drop to their final level one or two repeats later. (b) With two seconds of sustained load before timing, all repeats agree.

with no message. The cause is a watchdog on the cluster that cancels a GPU job once it is about 30 minutes old if its mean GPU utilization is 5% or less. A configuration sweep is exactly such a job: more than 85% of its wall time is TensorRT building engines, which is mostly CPU work, and timing a sub-millisecond engine for 100 iterations keeps the GPU busy for a tenth of a second. We did not add artificial load to stay above the threshold. The harness was already resumable, so the second sweep was submitted as a chain of jobs, each starting when the previous one ends and continuing from the same file.

2) A 20% timing artifact: For 30 of the 79 successful runs, the three repeats of the *same engine* disagreed by more than 8%. The disagreement has a clear shape (Fig. 1a): the first one or two repeats are 20% to 25% slower, then the latency drops and stays down. It never goes the other way. It affects only configurations whose latency is set by the GPU, namely the single engine and everything run under CUDA Graphs; eager PyTorch and the configurations with PyTorch fallback blocks, whose latency is set by the CPU launching kernels, are stable to within 4%.

Our reading is that the GPU needs a few hundred milliseconds of sustained load to reach full speed after sitting idle through a compile, and that the 20-iteration warm-up is over long before that when an iteration takes 0.6 ms. We did not confirm the mechanism (clock or power state) directly, so we treat it as an observation about this node. The effect on a selector study is severe either way. The differences we are trying to resolve on the unmodified model are of the order of 10%, and with three repeats the median lands on the slow or the fast level essentially at random. Under these conditions the selector would have been fitted to noise, and it would have looked as if the best configuration changed from shape to shape. That steady-state measurement is easy to get wrong without doing anything obviously wrong is well documented [8], [9]; this is one more instance.

E. Iteration 3: The Corrected Sweep

The second sweep differs from the first in three ways. Before timing, the harness now keeps the GPU busy with the workload itself for two seconds; the shared benchmark routine then runs unchanged, including its own 20 warm-up calls. Each module is timed five times instead of three, and the latency we use is the median of the five medians. And every row records the node and job it came from, in case a chained sweep ends up spread over several machines. The first sweep's data was discarded.

After two more failed node boots, the third job of the chain ran all 340 pairs in one piece, in 2 h 31 min on a single node. The warm-up had an unplanned side effect: it lifted the job's GPU utilization above the watchdog threshold, so the chain was never needed.

The data is clean. Of the 340 pairs, 320 succeeded. The 20 failures are all `ln_global`, which fails to build on every shape with an internal error in Torch-TensorRT 2.14 (a missing `_frozen_param0` attribute on the partitioned graph); it is excluded from selection. Across the five repeats the median spread is 0.7%, the 90th percentile 2.4% and the maximum 5.8% (Fig. 1b). The default configuration and its rebuild differ by 0.3% in the median. We therefore treat differences below about 1% as ties. Every compiled module agrees with eager PyTorch to a cosine similarity of at least 0.99999, with a largest absolute difference of 0.02 in FP16.

F. Selectors

Both selectors are deliberately small. With twelve training shapes there is no room for anything else.

The *lookup selector* stores, for each training shape, the configuration that was fastest there, and answers a query with the entry of the nearest training shape in $(\log_2 b, \log_2 s)$. It has no parameters and cannot extrapolate.

The *regression selector* fits, for each configuration c, the latency model

$$\hat{L}_c(b,s) \;=\; \theta_{c,0} + \theta_{c,1}\, b + \theta_{c,2}\, bs + \theta_{c,3}\, bs^2, \qquad \theta_c \geq 0, \quad (2)$$

whose terms stand for a fixed per-call overhead, a per-sequence cost, a per-token cost and the quadratic cost of self-attention. The coefficients are obtained by non-negative least squares [10] on the *relative* error, $\sum_w (\hat{L}_c(w)/L(c,w) - 1)^2$, so that small and large shapes carry equal weight. Non-negativity keeps the model physically meaningful and prevents it from predicting negative latencies outside the grid. The selector returns $\arg\min_c \hat{L}_c(b,s)$.

We evaluate in two ways. *Held-out*: fit on the twelve grid shapes, test on the eight off-grid shapes. *Leave-one-out*: for each grid shape, fit on the other eleven and test on the one left out. To relate quality to tuning effort, we also refit every method on subsets of $k = 1, \ldots, 12$ grid shapes (all subsets where there are at most 40, otherwise 40 random ones) and test on the held-out shapes.

G. Results

1) The latency landscape: Fig. 2 shows the structure that everything else follows from. Eager PyTorch takes between 3.2 and 4.1 ms on every shape below $(16, 448)$, almost independent of size: at these sizes an FP16 DistilBERT forward pass on an A100 is bounded by the time it takes Python and the framework to issue a long sequence of small kernels, not by the kernels themselves. The single TensorRT engine removes that overhead and is 2.68× faster than eager in the geometric mean over all 20 shapes, from 6.6× at $(1, 128)$ down to 1.07× at $(16, 512)$, where the GPU is finally the bottleneck.

A partitioned model sits in between, and where it sits depends on how many times execution returns to PyTorch. With GELU excluded (six fallback blocks) the default configuration is 1.63× slower than the single engine; with LayerNorm excluded (seven) it is 2.49× slower and only 7% faster than not compiling at all. Table IV summarizes what each control is worth.

Two things stand out. First, the partitioning controls barely matter as long as the engines survive: `min_block_size` and the partitioner move latency by 5% or less, and their only large effect is the destructive one, when a high threshold discards engines and pushes the model back towards eager. Second, one runtime option dwarfs all of them. CUDA Graphs makes the GELU-partitioned model 1.63× faster and the LayerNorm-partitioned one 2.26× faster, to within 11% and 23% of the single engine under the same option. This is consistent with the picture above: what a fallback block costs is launch overhead, and replaying a recorded launch sequence removes it no matter how many blocks there are.

Fig. 3 shows the same thing from the point of view of tuning. The *size* of the gain is strongly workload dependent, from 5.0× at $(1, 128)$ with LayerNorm excluded down to a few percent at $(16, 512)$. The *identity* of the best configuration is not, and only the latter is something a selector can exploit.

TABLE IV
EFFECT OF EACH CONTROL ON LATENCY (GEOMETRIC MEAN OVER ALL 20 SHAPES, RANGE OVER SHAPES IN PARENTHESES)

Change	Latency
Unmodified model, relative to default	
Optimization level 5	1.00× (0.97–1.02)
Optimization level 1	1.06× slower (1.03–1.08)
CUDA Graphs	1.11× faster (1.02–1.20)
GELU in PyTorch, relative to its default	
`min_block_size` 1 (7 engines)	1.01× faster (0.99–1.04)
Global partitioner	1.02× slower (0.98–1.05)
`min_block_size` 22 (1 engine)	1.74× slower (1.07–2.25)
CUDA Graphs	1.63× faster (1.06–3.27)
LayerNorm in PyTorch, relative to its default	
`min_block_size` 1 (13 engines)	1.05× faster (0.99–1.11)
`min_block_size` 10 (6 engines)	1.03× slower (1.00–1.05)
`min_block_size` 20 (no engine)	1.28× slower (1.04–1.47)
CUDA Graphs	2.26× faster (1.08–5.02)

TABLE V
HELD-OUT SHAPES: REGRET R FROM (1) AND SPEEDUP OVER DEFAULT TORCH-TENSORRT. GELU AND LAYERNORM DENOTE THE SCENARIOS IN WHICH THAT OPERATOR RUNS IN PYTORCH.

	Unmodified		GELU		LayerNorm	
Method	R (%)	speedup	R (%)	speedup	R (%)	speedup
PyTorch eager	209.8	0.36×	180.9	0.57×	153.8	0.92×
Default TRT	11.0	1.00×	61.3	1.00×	132.7	1.00×
Fixed tuned	0.0	1.11×	0.0	1.61×	0.5	2.32×
Lookup sel.	0.0	1.11×	3.2	1.56×	1.4	2.29×
Regression sel.	1.5	1.09×	0.5	1.60×	0.0	2.33×
Best tested	0.0	1.11×	0.0	1.61×	0.0	2.33×

2) Selection on held-out shapes: Table V and Fig. 4 give the main result. In all three scenarios the fixed tuned configuration is the CUDA Graphs variant, and it is the best tested configuration on 23 of the 24 scenario–shape combinations. Its regret is 0.0%, 0.0% and 0.5%. The selectors range from 0.0% to 3.2%; relative to the fixed tuned configuration they achieve between 0.97× and 1.01×. The single case in which a selector is ahead (regression, LayerNorm scenario, by 0.5%) is below the noise floor. Leave-one-out evaluation on the grid agrees: the fixed tuned configuration has a regret of 0.0%, 0.1% and 0.9%, the lookup selector makes identical choices, and the regression selector scores 0.8%, 1.0% and 0.0%.

Against default Torch-TensorRT, on the other hand, the gain is real and sizeable: 1.11× on the unmodified model, 1.61× with GELU excluded and 2.32× with LayerNorm excluded.

3) Why there is nothing to select: On the unmodified model, `trt_cudagraphs` is the fastest configuration on all 20 shapes. In the partitioned scenarios the CUDA Graphs variant is fastest everywhere except at the top-right corner of the shape space (Fig. 5): at $(16, 512)$ in both scenarios and at $(16, 448)$ with LayerNorm excluded, eager PyTorch is faster by 1% to 11%, because at that size the per-call overhead no longer matters and the compiled module pays for crossing between engines and PyTorch.

That one crossover is the entire workload dependence in

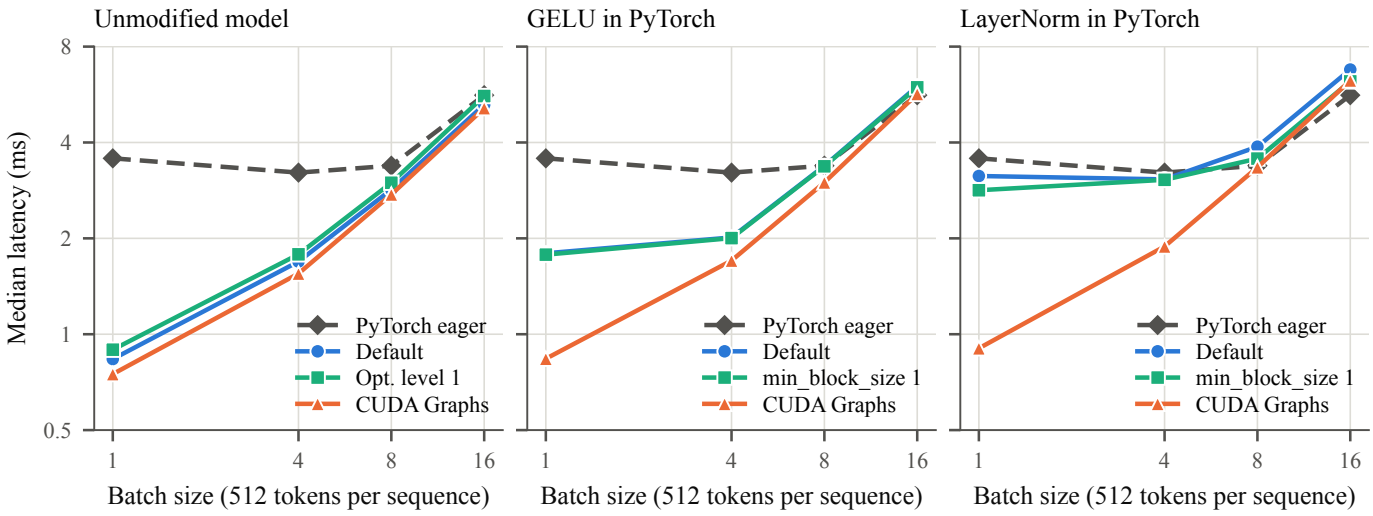


Fig. 2. Median latency against batch size at 512 tokens. Eager PyTorch is flat until the largest batch, which shows that it is limited by per-call overhead and not by GPU work. A partitioned model inherits part of that overhead (circles, squares); CUDA Graphs removes it (triangles). All curves meet at batch 16, where GPU work dominates.

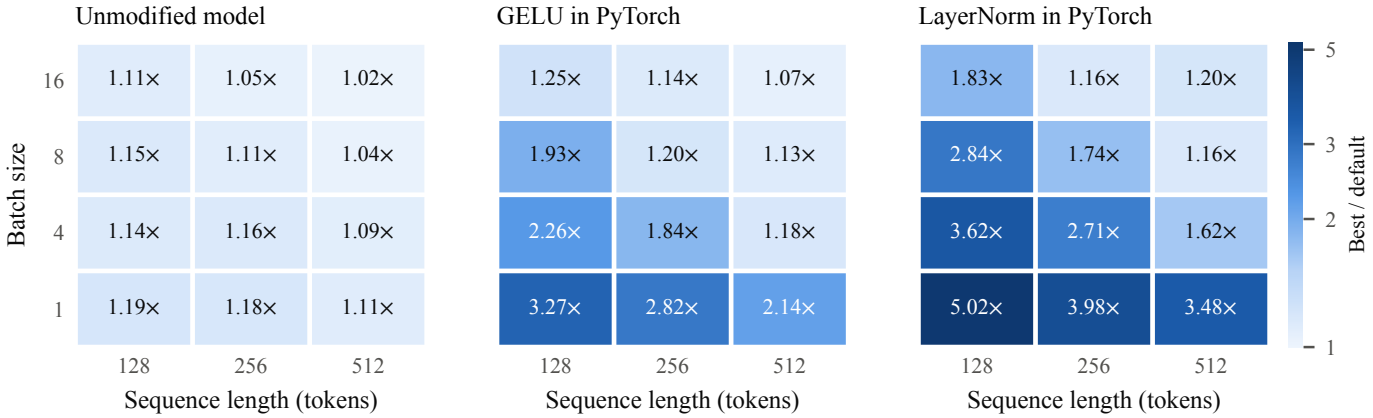


Fig. 3. Speedup of the best tested configuration over the scenario's default on the training grid. Tuning is worth most at small shapes and when the graph is partitioned; it fades as the GPU becomes the bottleneck.

our data, and the selectors handle it poorly. Across the three scenarios the two selectors chose eager for a held-out shape seven times, and it was the right call twice. The lookup selector assigns $(12, 384)$ to its nearest grid neighbor $(16, 512)$ and therefore to eager, at a cost of 23% on that shape with GELU excluded and 12% with LayerNorm excluded. The regression selector does better on the partitioned scenarios but sends $(16, 448)$ to eager on the unmodified model, where the engine is 13% faster. The underlying problem is that the boundary is defined by a single training point at the edge of the grid, and that the latency model is least accurate for exactly the configuration on the other side of it: its mean absolute error on held-out shapes is 3% to 4% for the GPU-bound configurations but 13% for eager, whose flat-then-rising curve the four terms of (2) do not fit well.

4) Tuning cost: The cost of a measurement is the engine build, the first call, the warm-up and the timing. Measuring every configuration of a scenario on all twelve grid shapes costs 17 to 33 GPU-minutes (Table VI), of which engine builds are the bulk: a single-engine build takes 8 to 14 s at the default optimization level and 32 s at level 5, a partitioned one about 30 s, and the 13-engine `ln_mbs1` about 57 s. Fig. 6 shows that most of this effort is unnecessary. Two measured shapes identify the fixed tuned configuration in every scenario, and the ten further shapes buy nothing for it; the selectors need

TABLE VI
TUNING COST AND BREAK-EVEN AGAINST DEFAULT TORCH-TENSORRT, PER SCENARIO

	Unmodified	GELU	LayerNorm
Saving per request (ms)	0.12	0.70	1.76
Full grid, 12 shapes (GPU-min)	16.6	30.1	33.3
break-even (requests)	8.3×10^6	2.6×10^6	1.1×10^6
Two shapes (GPU-min)	2.7	5.1	5.5
break-even (requests)	1.4×10^6	4.3×10^5	1.9×10^5

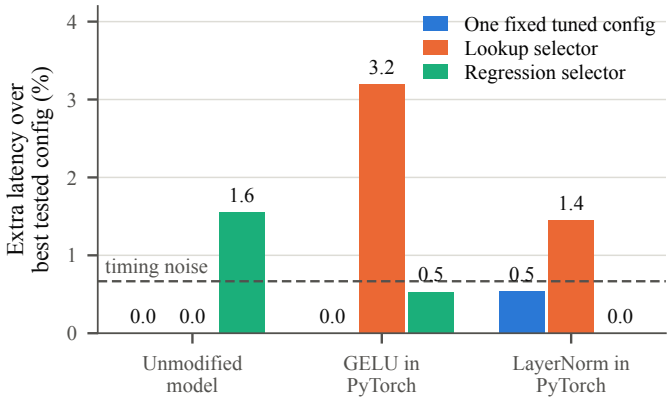


Fig. 4. Regret on the eight held-out shapes for the three methods that use tuning data. The dashed line is the timing noise (0.7%). No selector beats the fixed tuned configuration by more than the noise.

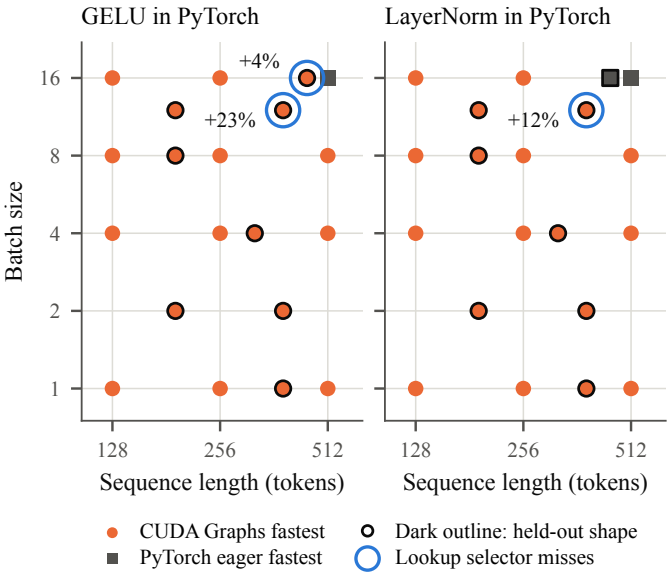


Fig. 5. Fastest configuration at each of the 20 shapes in the two partitioned scenarios. The only shape dependence is a switch to eager PyTorch at the largest shapes. Circles mark held-out shapes where the lookup selector chose differently from the best configuration, with the resulting loss.

more data and still end up level or behind. Dividing the cost by the mean saving per request on the held-out shapes gives the number of requests after which tuning has paid for itself: about 1.4 million on the unmodified model, where the saving is 0.12 ms, and 0.2 to 0.4 million in the partitioned scenarios.

H. Discussion

The proposal's hypothesis for this experiment was that a lookup table or a regression model would recover most of the gap between a fixed configuration and the best one. For this model the hypothesis fails for an uninteresting reason: the gap is not there. We think the reason generalizes somewhat beyond our setup, and it is worth stating.

A selector needs configurations whose latency curves *cross*. In our space, the configurations differ in how much per-call overhead they carry, and overhead is a constant offset: a configuration with less of it is faster at every shape, by an amount that shrinks as the shape grows but does not change sign. CUDA Graphs removes the most overhead and therefore wins everywhere it is applicable. The only crossing we observe involves eager PyTorch at the largest shapes, and it is shallow (at most 11%) and located at the edge of the measured region. Per-workload selection becomes interesting when configurations trade a fixed cost against a marginal one, for instance precision modes with different kernel speeds, engines built with dynamic versus static shape profiles, or models where a fallback block does real computation instead of just interrupting the engine. None of these is in our space.

For the project as a whole, three findings from this experiment matter more than the selector:

1) DistilBERT converts to a single TensorRT engine. Any partitioning effect studied on it has to be induced.
2) On this model the cost of a partition boundary is launch overhead, and CUDA Graphs recovers most of it. Whether a graph is "well partitioned" matters far less with that option enabled.
3) The shared benchmark's 20-iteration warm-up is too short for sub-millisecond engines on this hardware and overstates their latency by about 20% in some repeats. Experiments that compare a single engine against PyTorch with it will understate the speedup.

I. Threats to Validity

The partitioned scenarios are artificial. GELU and LayerNorm convert without difficulty, and nobody would exclude them in practice. The scenarios reproduce the structure of a fallback (execution leaves the engine and comes back) but not necessarily its cost profile, since a genuinely unsupported operator may be expensive in itself.

One model, one GPU, one precision. Everything here is DistilBERT in FP16 on one A100 node, with sequences of at most 512 tokens. All 340 pairs were measured in a single job on a single machine, which removes machine-to-machine variation from the comparison but also means we have not observed it.

Static shapes. Every engine is built for exactly one shape. A deployment that serves many shapes from one engine with a dynamic profile faces a different trade-off, which this experiment does not measure.

Drift in the CPU-bound baselines. The held-out shapes were measured in the last hour of the job, and the CPU-bound configurations look slower there than on the grid. Relative to the grid-fitted model (2), eager PyTorch is 15% slower on the held-out shapes on average, the LayerNorm-partitioned configurations 8% to 11%, and the GPU-bound configurations 1%. Some of this is model error, but neighboring shapes point the same way (eager takes 3.25 ms at $(8, 256)$ on the grid and 4.06 ms at $(8, 192)$ held out), so host-side performance probably degraded during the job. Two consequences follow. The held-out speedups over eager and over the partitioned defaults are somewhat inflated; the leave-one-out figures on

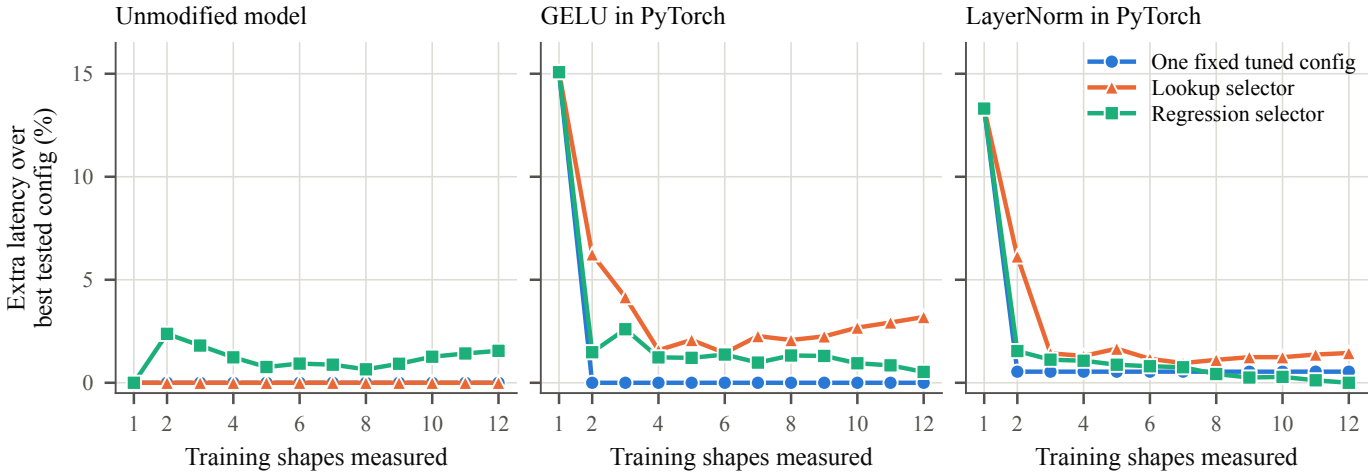


Fig. 6. Held-out regret when only k of the twelve grid shapes are measured (mean over subsets). Two shapes are enough to identify the fixed tuned configuration. With one shape the outcome depends on which: if it is $(16, 512)$, every method selects eager PyTorch.

the grid (2.22× over the default for LayerNorm, against 2.32× held out) are the more conservative ones. And the position of the eager crossover on the held-out shapes is uncertain: if eager were in fact 15% faster at $(16, 448)$, it would be the best choice there in all three scenarios, the fixed tuned configuration would lose 2% to 20% on that one shape, and a selector choosing eager there would be right. This would move the held-out regret of the fixed tuned configuration to at most 2.3%. It would not change the picture at the other seven shapes or on the grid.

Build times share a cache. TensorRT reuses layer timings within a process, so the first build of the job took 40 s and later builds of the same graph 6 to 13 s. The tuning costs in Table VI are therefore optimistic for a cold start.

Memory. The recorded peak memory counts PyTorch's allocator only and excludes memory held by TensorRT engines, so we do not report a memory comparison.

J. Summary

We built a resumable sweep harness, two workload-aware selectors and an evaluation on held-out shapes, and measured 17 configurations on 20 shapes. For DistilBERT on an A100, tuning the configuration once is worth 1.11× to 2.32× over default Torch-TensorRT, two measured shapes are enough to do it, and selecting the configuration per workload adds nothing beyond that. The result rests on a measurement protocol that had to be corrected first; with the original warm-up the same experiment would have reported a workload-dependent optimum that does not exist.

References

[1] J. R. Rice, "The algorithm selection problem," in *Advances in Computers*. Elsevier, 1976, vol. 15, pp. 65–118.

[2] T. Chen, L. Zheng, E. Yan, Z. Jiang, T. Moreau, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Learning to optimize tensor programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[4] PyTorch Contributors, "Torch-TensorRT documentation," https://pytorch.org/TensorRT/, accessed Nov. 2026.

[5] NVIDIA Corporation, "NVIDIA TensorRT documentation," https://docs.nvidia.com/deeplearning/tensorrt/, accessed Nov. 2026.

[6] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[7] A. Gray, "Getting started with CUDA graphs," NVIDIA Technical Blog, 2019.

[8] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2007, pp. 57–76.

[9] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009, pp. 265–276.

[10] C. L. Lawson and R. J. Hanson, *Solving Least Squares Problems*. SIAM, 1995.